

Ansible Configuration Management

Category: **Configuration Management & Orchestration** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Ansible Configuration Management**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- **Agentless Architecture & SSH Multiplexing:** Ansible operates without daemons or agents on target hosts. It connects via OpenSSH, transfers lightweight ephemeral Python modules to `/tmp`, executes them, and returns structured JSON results.

Production Application: Significantly simplifies management and security compliance because no custom agent requires patching or port listening on target servers.

- **Idempotency & Desired State Enforcement:** Tasks are designed such that applying them repeatedly ensures the target host matches the declared desired state without making redundant modifications or triggering unintended side effects.

Production Application: Ensures playbooks can be safely run continuously every hour to automatically remediate configuration drift without restarting healthy services.

- **Playbook Handlers & Change Notification:** Handlers are specialized tasks that execute only once at the end of a play, and only if another task explicitly reported a `changed` state and notified the handler.

Production Application: Prevents redundant service restarts when updating configuration files (e.g. Nginx or PostgreSQL is restarted only when configuration content actually changed).

- **Ansible Vault Encryption & Credential Isolation:** Symmetric AES-256 encryption mechanism for securing sensitive variables, passwords, and private keys directly within source control.

Production Application: Allows teams to commit encrypted `vault.yml` files directly to Git while keeping plaintext secrets protected behind vault passwords in CI/CD.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
ansible-playbook -i inventories/production/hosts.yml site.yml --check --diff ansible-playbook [options]	Execute an Ansible playbook in dry-run simulation mode showing exact unified diff of proposed file modifications without making live changes. Options: [{ 'flag': '-i ', 'description': 'Specify target inventory path or dynamic inventory script.' }, { 'flag': '--check', 'description': 'Dry-run mode: do not make any live changes on remote hosts.' }, { 'flag': '--diff', 'description': 'Show unified diff of changes that would be made to configuration files.' }, { 'flag': '--limit ', 'description': 'Restrict execution to a specific subset of hosts or groups.' }]	Always run with `--check --diff` during maintenance windows to preview modifications before live application. Pitfall: Relying solely on `--check` for tasks that depend on earlier tasks (e.g. checking a package install when later tasks expect the binary to exist).
ansible all -i inventory.ini -m ping -o ansible -i -m ping [options]	Verify SSH reachability and valid Python interpreter execution across all inventory hosts with compact single-line output. Options: [{ 'flag': 'all', 'description': 'Target all hosts defined in the inventory.' }, { 'flag': '-m ping', 'description': 'Invoke the Ansible ping module (tests SSH + Python, not ICMP).' }, { 'flag': '-o', 'description': 'Format output as a compact one-liner per host.' }]	Run during initial cluster provisioning and automated infrastructure health checks. Pitfall: Assuming `ansible -m ping` is an ICMP network ping; it actually validates SSH key authentication and Python execution.
ansible-vault encrypt_string --vault-id prod@prompt 'SuperSecretDbPassword123' --name 'db_password' ansible-vault encrypt_string [options] "	Encrypt a single sensitive string inline using AES-256 encryption with a named vault ID for embedding in plaintext variable files. Options: [{ 'flag': '--vault-id ', 'description': 'Use a specific vault password ID and source (prompt, file).' }, { 'flag': "--name '", 'description': 'Assign the encrypted string to a variable name.' }]	Embed encrypted variable strings directly inside standard YAML files so non-sensitive variables remain easily readable and diffable. Pitfall: Committing unencrypted vault password files or forgetting vault passwords, rendering encrypted files unrecoverable.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
ansible-galaxy role install geerlingguy.nginx -p ./roles --force ansible-galaxy role install [options]	Download and install a community or enterprise reusable role from Ansible Galaxy into a local project roles directory. Options: [{ 'flag': '-p ', 'description': 'Target directory path to install the role.' }, { 'flag': '--force', 'description': 'Overwrite existing installed roles.' }, { 'flag': '-r requirements.yml', 'description': 'Install all roles and collections declared in a requirements file.' }]	Define all external roles and collections in a version-pinned `requirements.yml` file and install during build. Pitfall: Not pinning role versions in `requirements.yml`, leading to breaking changes when upstream authors update roles.
ansible-lint site.yml ansible-lint [options] [playbook.yml]	Perform comprehensive static analysis on playbooks and roles to enforce best practices and detect common syntax errors. Options: [{ 'flag': '-v', 'description': 'Verbose output detailing rule evaluations.' }, { 'flag': '--profile=production', 'description': 'Enforce strict production ruleset including idempotency and naming standards.' }]	Execute in pre-commit hooks and CI pipelines to prevent broken playbooks from being merged into main. Pitfall: Ignoring lint warnings about missing task names (`name:`), making debugging failed playbook runs extremely difficult.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.